RAG - BASED CUSTOMER SUPPORT ASSISTANT

Low-Level Design Document

Prepared by: Rahul Pravin Dhumal
Intern ID: IN226095102

Document Type	Low-Level Design (LLD)
Project Title	RAG-Based Customer Support Assistant
Modules	(loader, embedder, retriever, graph, hitl, memory, main, config)
Workflow Engine	LangGraph (3-node graph) Process Node → Answer Node / HITL Node
API	FastAPI — POST /ask

1.Module-Level Design

The system is organized into 8 distinct modules, each with a single clear responsibility. This modular design ensures separation of concerns, easy unit testing, and the ability to swap individual components (e.g., the LLM provider or vector database) without affecting the rest of the system.

1.1 Folder Structure

```
rag_customer_support/

├── app/
│   ├── __init__.py          # Marks app as a Python package
│   ├── loader.py            # PDF loading + text chunking
│   ├── embedder.py          # Embedding generation + ChromaDB storage
│   ├── retriever.py         # Similarity search + reranking
│   ├── graph.py             # LangGraph workflow (process, answer, hitl nodes)
│   ├── hitl.py              # Human-in-the-Loop escalation (queue logging)
│   ├── memory.py            # Custom in-memory conversation buffer
│   ├── main.py              # FastAPI entry point (API endpoints)
│   └── config.py            # Configuration (API keys, paths, model)

├── data/
│   └── customer_support_manual.pdf   # Knowledge base document

├── chroma_db/               # Vector database storage (auto-generated)

├── human_review_queue.txt   # HITL escalation logs

├── requirements.txt         # Python dependencies

├── .env                     # Environment variables (Groq API key)

├── .gitignore               # Ignore venv, DB, secrets

└── README.md                # Project documentation
```

This folder structure follows a modular design approach, separating concerns such as data processing, retrieval, workflow orchestration, and API handling. It ensures scalability, maintainability, and ease of debugging.

1.2 Module Descriptions

1. Document Processing Module (loader.py)	**Responsibility:** • Load PDF • Convert into raw documents **Key Function:** load_and_chunk ()
2. Chunking Module	**Responsibility:** • Split text into smaller chunks • Maintain overlap for context **Technique Used:** RecursiveCharacterTextSplitter
3. Embedding Module (embedder.py)	**Responsibility:** • Convert chunks → vectors • Store in vector DB **Model Used:** HuggingFace (MiniLM)
4. Vector Storage Module	• Uses ChromaDB • Stores embeddings • Enables similarity search
5. Retrieval Module (retriever.py)	**Responsibility:** • Fetch top-k relevant chunks • Calculate similarity score • Apply reranking
6. Query Processing Module	• Handles user query • Passes to graph workflow
7. Graph Execution Module (graph.py)	• Built using LangGraph • Controls workflow • **Nodes :** process node , answer node ,hitl node
8. HITL Module (hitl.py)	**Responsibility:** • Handle escalation • Store queries in queue
9. Memory Module (memory.py)	**Responsibility:** • Store conversation history • Provide context for multi-turn queries
10. API Layer (main.py)	• Built using FastAPI • Handles user requests

2. Data Structures

The following data structures are used throughout the system. Consistent schemas enable reliable data flow between modules.

2.1 Document Chunk

```
{
 "page_content": "text chunk",
 "metadata": {}
}
```

2.2 Embedding

```
[0.12, 0.98, 0.45, ...]
```

2.3 Query-Response Schema

```
{
 "query": "user question",
 "answer": "generated response"
}
```

2.4 Graph State Object

```
{
 "query": str,
 "context": str,
 "route": str,
 "response": str
}
```

3. Workflow Design (LangGraph)

3.1 Graph Structure

```
START
↓
process_node
↓
decision

├── answer_node
└── hitl_node
↓
END
```

3.2 Node Responsibilities

Process_Node

- Retrieve Context
- Calculate Similarity Score
- Detect Intent
- Decide Route

Answer_Node

- Generate Answer Using Llm
- Use Memory + Context

Hitl_Node

- Escalate Query
- Log To File

4. Conditional Routing Logic

The routing layer is the brain of the system. It operates inside process_node and determines whether a query is confident enough to be answered by the LLM, or should be escalated to a human agent.

4.1 Routing Factors

Intent Detection	Classifies the query into: order, refund, account, payment, or unknown using keyword matching on the query string.

Similarity Score	Average cosine similarity of the top-k retrieved chunks. High score = good match. Low score = poor retrieval = lower confidence.

Memory Presence	If a prior conversation turn exists in memory, the system has more context and can attempt an answer even with borderline confidence.

4.2 Intent Categories

Intent	Trigger Keywords	Routing Bias
order	order, tracking, delivery, shipment, shipped	**Answer (if score sufficient)**
refund	refund, return, money back, cancel, reimburse	**Answer (if score sufficient)**
account	account, login, password, profile, email	**Answer (if score sufficient)**
payment	payment, billing, charge, invoice, card	**Answer (if score sufficient)**
unknown	anything not matching above patterns	**HITL (escalate)**

4.3 Routing Decision Logic

```
def process_node(state):

    query=state["query"].lower()

    result=retrieve_context(query)

    context=result["context"]

    score=result["score"]

    if "password" in query or "account" in query:
        intent="account"

    elif "order" in query or "shipping" in query:
        intent="order"

    elif "refund" in query or "return" in query:
        intent="refund"

    elif "payment" in query or "deducted" in query:
        intent="payment"

    elif "arrive" in query or "delivery" in query:
        intent="order"

    else:
        intent="unknown"

    memory = get_memory_context()

    if intent=="unknown" and memory.strip()==="":
        return {
           "query":query,
           "context":context,
           "route":"hitl",
           "response":""
        }

    if score > 1.8 and memory.strip()==="":
        return {
            "query":query,
            "context":context,
            "route":"hitl",
            "response":""
        }

    return {
        "query":query,
        "context":context,
        "route":"answer",
        "response":""

    }
```

4.4 Routing Decision Summary

CONDITION intent == 'unknown' AND memory empty	**ROUTE TO** **→ HITL Node**
CONDITION score > 1.8 AND memory empty	**ROUTE TO** **→ HITL Node**
CONDITION Score ≤ 1.8OR memory has prior turns	**ROUTE TO** **→ Answer Node (LLM)**

5. HITL (Human-in-the-Loop) Design

The HITL module ensures that the system degrades gracefully when it lacks confidence. Rather than generating a potentially incorrect answer, it routes the query to a human agent for review.

5.1 Trigger Conditions

Unknown Intent	Unknown intent is escalated only if no prior conversation context exists.If memory is present, the system attempts to answer using context.
Low Confidence	Average similarity score of retrieved chunks falls below SIMILARITY_THRESHOLD (default 0.5).
Missing Context	No relevant chunks retrieved from ChromaDB — context string is empty or too short.

5.2 HITL Flow

```
User Query
    |
    ▼
process_node sets route = 'hitl'
    |
    ▼
hitl_node executes:
  1. Format log entry (timestamp + query + intent + score + reason)
  2. Append to human_review_queue.txt
  3. Return escalation message to user
    |
    ▼
Response: "Your query has been escalated to our support team."
    |
    ▼
Human agent reads human_review_queue.txt and responds manually
```

5.3 HITL Module Code Sketch

```
def escalate_to_human(query):

    with open(
       "human_review_queue.txt",
       "a",
       encoding="utf-8"
    ) as file:

        file.write(
          f"""
Time: {datetime.now()}
Query: {query}
Status: Pending Human Review
------------------------
"""
        )

    return f"""
Escalation Triggered.

Ticket added to Human Review Queue.

Support team will review:
{query}
"""
```

6. API / Interface Design

The system exposes a single REST endpoint via FastAPI. All user interaction passes through this endpoint. Swagger UI is auto-generated and accessible at /docs for testing during development

6.1 Endpoint Specification

Method	POST
Path	/ask
Content-Type	application/json
Auth	None (development mode)
Description	Submit a customer support query and receive an answer or escalation response

6.2 Request Schema

```
POST /ask
Content-Type: application/json

{
    "query": "What is refund policy?"
}
```

6.3 Response Schema - Successful Answer

```
HTTP 200 OK
{
    "query":  "What is the refund policy for damaged items?",
    "answer": "Refunds are processed within 5-7 business days."}
```

6.4 Response Schema — HITL Escalation

```
HTTP 200 OK
{
    "query":  "How does your enterprise SLA work?",
    "answer": " Escalation Triggered.
               Ticket added to Human Review Queue.”}
```

6.5 Interaction Flow

```
Client                    FastAPI                LangGraph
  |                          |                      |
  |── POST /ask ────────────▶ |                      |
  |   {query: '...'}         |                      |
  |                          |── invoke(state) ──▶ |
  |                          |                      |── process_node
  |                          |                      |── answer_node /
hitl_node
  |                          |◀── state (response) ─|
  |◀── 200 {answer: '...'} ─|                      |
```

7. Error Handling

The system handles multiple failure modes gracefully, ensuring users always receive a response rather than an unhandled exception.

Error Scenario	Likely Cause	Handling Strategy
No relevant chunks found	PDF not ingested or query too vague	Return escalation message; route to HITL
Low confidence retrieval	Query topic not covered in KB	Similarity score below threshold → HITL route
LLM API failure (Groq)	Network timeout or quota exceeded	Catch exception; return fallback message; log error
Unknown / off-topic query	User asks outside support domain	Intent = unknown → HITL; logged to queue
ChromaDB read error	Corrupt collection or missing file	Raise startup exception with clear error message
Empty query string	User submits blank request	FastAPI Pydantic validation returns 422 before processing

7.1 Fallback Response Strategy

```
def answer_node(state):

    memory = get_memory_context()

    prompt = f"""
    You are a professional customer support assistant.
    Rules:
    - Answer ONLY using the provided context
    - Be clear and concise
    - If unsure, say you are not confident

    Previous Conversation:
    {memory}

    Context:
    {state['context']}

    Question:
    {state['query']}

    Answer:
    """

    try:
        result = llm.invoke(prompt)

        save_to_memory(
            state["query"],
            result.content
        )

        return {
            "response": result.content
        }

    except Exception as e:
        print(f"LLM Error: {e}")  # you can use logging later

        return {
            "response": "We are experiencing technical difficulties. Please try again later."
        }
```

8. LLD Summary

This Low-Level Design document defines the complete internal architecture of the RAG Customer Support Assistant, specifying how each module is implemented, how data flows between them, and how error conditions are handled.

Total Modules	loader, embedder, retriever, graph, hitl, memory, main, config
Data Structures	Document Chunk, Embedding Vector, GraphState, QueryRequest, QueryResponse, HITL Queue Entry
Graph Nodes	process_node → [answer_node | hitl_node] → END
Routing Logic	Based on intent (5 categories), similarity score (threshold 0.5), and memory presence
HITL Trigger	Unknown intent + no memory OR low confidence score + no memory
API	POST /ask — FastAPI with Pydantic validation, auto-Swagger at /docs
Error Handling	6 error scenarios covered with fallback responses and HITL routing
Design Principle	Modular, single-responsibility, swap-friendly (LLM / vector DB / embedding model)